5.1.3.3 K-Nearest Neighbor o k vecinos más cercanos

También conocido como **KNN** por sus siglas en inglés.

El algoritmo no intenta extraer una característica de los datos haciendo un proceso de entrenamiento exhaustivo con ellos. Simplemente los almacena para recuperarlos en el futuro.

Cuando se presenta un nuevo caso, éste es comparado con la base de casos y se extrae aquel más similar (o los K más similares) para comprobar cual era la clase a la que pertenecía. Se asume que si el caso es similar, la solución será similar.

Se necesita definir una función de similitud entre dos ejemplos. La calidad de esta función de similitud es crítica para éste algoritmo. Si la función de similitud no es capaz de medir correctamente la similitud que existe entre dos casos dados, la asunción de que a similares casos se espera que la instancia sea de la misma clase pierde vigencia y KNN deja de clasificar correctamente.

La solución a seleccionar depende del K elegido. Si K = 1 será la instancia recuperada más similar. Si K > 1 habrá que tomar una decisión de qué clase elegimos. Normalmente el criterio más lógico es **la clase mayoritaria**. Si la salida es un número real, podríamos hacer la media.

Hay multitud de **medidas de distancia** que podemos usar:

- **Distancia Euclídea**: Para cada atributo de X e Y

$$D - Euclidea(X, Y) = \sqrt[2]{\sum_{i=1}^{N} (x_i - y_i)^2}$$

- **Distancia de Manhattan**: Para cada atributo de X e Y

$$D - Man(X, Y) = \sum_{i=1}^{N} |x_i - y_i|$$

- **Distancia de Minkowski**: generalización de la euclídea o la de manhattan donde el exponente y la raíz pueden ser cualquier número.

$$D - Minkowski(X, Y) = \sqrt[p]{\sum_{i=1}^{N} (x_i - y_i)^p}$$

- **Distancia de edición**: el número de cambios que hay que hacer para convertir una instancia en otra. Generalización de Hamming.
  - casa → cala (sustitución de 's' por 'l') = 1
  - casa → calle (sustitución de s por L, sustitución a por e e inserción de l) = 3

- **Distancia de Mahalanobis**: la similitud entre dos variables aleatorias multidimensionales. Se diferencia de la distancia euclídea en que tiene en cuenta la correlación entre las variables aleatorias.

Formalmente, la distancia de Mahalanobis entre dos variables aleatorias con la misma distribución de probabilidad $\vec{x}$ e $\vec{y}$ se calcula de la siguiente forma:

$$D - Mahalanobis(\vec{x}, \vec{y}) = \sqrt[2]{(x - \mu)^T * C^-1 * (x - \mu)}$$

Donde μ es la media de los datos y C-1 es la inversa de la matriz de covarianzas.

Ejemplo:

```
def mahalanobis(x=None, data=None, cov=None):
    """Computa la covarianza entre cada fila de los datos
    x    : Vector o matriz de datos con p columnas
    data : ndarray el total de datos de la distribución
    cov  : matriz de covarianzas (p x p) de la distribución. Si no se invoca se establece como none None
    """
    x_minus_mu = x - np.mean(data) #Cálculo de la media
    if not cov: #Calculamos la covarianza i
        cov = np.cov(data.values.T)
    inv_covmat = sp.linalg.inv(cov) # inversión de la matriz de covarianza
    left_term = np.dot(x_minus_mu, inv_covmat) # producto escalar entre x menos la media y la inversa de la matriz de covarianza
    mahal = np.dot(left_term, x_minus_mu.T)
    # producto escalar entre la traspuesta de x menos la media y el cálculo anterior
    return mahal.diagonal() # devolvemos la diagonal de la matriz
```

Nótese que en el ejemplo no calculamos la raiz cuadrada, por lo que en realidad estamos calculando el cuadrado de la distancía de mahalanobis

otra aproximación es ponderar el peso que tiene cada uno de los atributos en el cálculo de la distancia. Esto implica **introducir información del dominio** ya que debemos saber o intuir que ciertas variables aportan mas información a la hora de decidir cuales coger.

$$D - Euclidea - Ponderada(X, Y, P) = \sqrt[2]{\sum_{i=1}^{N} p_i (x_i - y_i)^2}$$

Donde P es el vector de pesos asociado a cada entrada.

En principio podemos ponderar cualquier distancia pero en unas tiene más sentido que en otras.

Ventajas de KNN

- No paramétrico (salvo que usemos distancias ponderadas). No hace suposiciones explícitas sobre la forma funcionar de los datos, evitando los peligros de la distribución subyacente de los datos.
- Algoritmo simple tanto de explicar como de interpretar.
- Alta precisión (relativa). Es bastante alta, aunque no superior a otros modelos más sofisticados. Pero a pesar de su aparente simpleza, si se elige correctamente la distancia puede ofrecer resultados bastante buenos
- El proceso de entrenamiento es inmediato

Inconvenientes del KNN

- Es muy sensible a los atributos irrelevantes. Hacer una buena selección de atributos relevantes es fundamental.
- Es sensible al ruido, ya que, si un ejemplo es un mal ejemplo de entrenamiento y es el seleccionado como el más similar, daremos una solución errónea. Esto se puede mitigar haciendo que K sea grande ya que reduce el ruido
- La ejecución es lenta si hay muchos datos de entrenamiento, ya que tiene que procesar todos los datos. Existen métodos para optimizarlo usando partición espacial pero aún asi es costoso
- Es caro en memoria ya que ocupa mucha memoria si hay muchos casos (Y tiene difícil solución, salvo limitar la memoria de trabajo)

Ejemplo de KNN en SKLearn

```
print(len(iris.data))
X_train = iris.data[:-20]
y_train = iris.target[:-20]
X_test = iris.data[-20:]
y_test= iris.target[-20:]

from sklearn.neighbors import KNeighborsClassifier
model = KNeighborsClassifier(n_neighbors = 1)

model.fit(X_train, y_train)

prediction = model.predict(X_test)
print("Predicción")
print(prediction)
print("Test")
print(y_test)

model.score(X_test, y_test)
```